Gas-Aware Implementation of Sponsored Fair Exchange

Semester Project at LASEC

Ryad Mohamed Aouak

Supervisor: Prof. Serge Vaudenay
EPFL, LASEC

Semester Project Presentation

A simple question

How can Alice buy a digital file from Bob

without either side having to trust the other?

If Bob sends first

Alice may keep the file and never pay.

If Alice pays first

Bob may keep the money and send nothing, or send a wrong file.

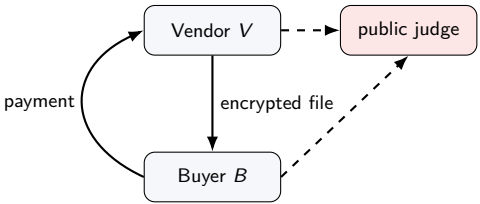
This is the fair-exchange problem.

Fair exchange target

Desired outcome

Either both parties get what they should, or the exchange is cancelled.

- The buyer receives an item matching a public description.
- The vendor receives the payment.
- No trusted human judge checks every file.



Why a smart contract appears

Mental model for this talk

A smart contract is a public, deterministic judge that can hold deposits and enforce payouts.

Good news

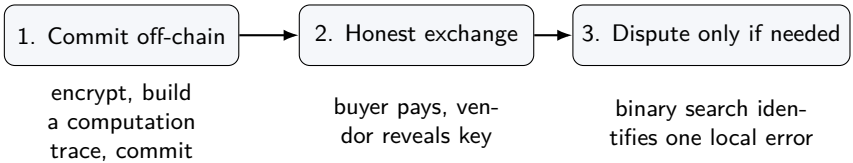
It can replace an always-online trusted third party.

Bad news

Every public computation and every deployment costs **gas**.

So the real engineering question is: what must be public, and what can stay local?

The SOX idea in one picture



Optimistic protocol

If everyone behaves honestly, the expensive verification never happens on-chain.

The cryptographic object behind SOX

Off-chain

The implementation builds a compact circuit/trace for:

- AES-CTR decryption;
- SHA-256 description check;
- commitments to ciphertext, circuit, and intermediate values.

On-chain

The contract stores short commitments and later verifies:

- membership proofs;
- one local gate computation;
- the final payout decision.

The file can be huge; the public judge should only see small authenticated pieces.

Baseline SOX flow

Optimistic path

1. V commits to the encrypted exchange data.
2. S funds the optimistic contract.
3. B pays.
4. V reveals the key.
5. If B accepts, V is paid.

Dispute path

1. B complains.
2. S_B and S_V fund the dispute.
3. A binary search narrows the disagreement.
4. One local step is verified on-chain.
5. The contract finalizes payouts.

My work keeps this goal but changes how it is implemented and measured.

What I inherited

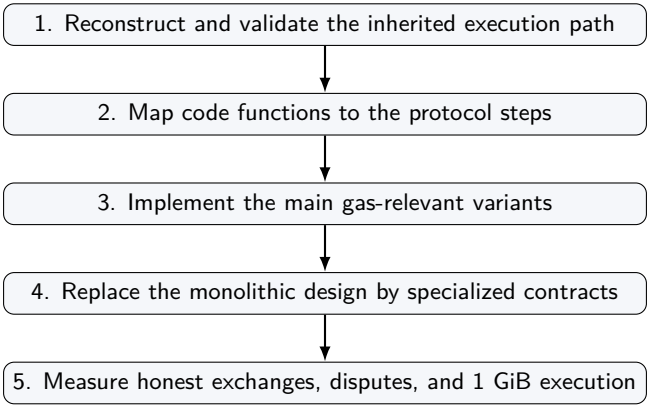
Already working

- Rust/WebAssembly precontract pipeline.
- Solidity optimistic and dispute contracts.
- Local web application and backend.
- Sponsored transaction infrastructure.

Main problems

- Expensive per-exchange deployments.
- No clean implementation of the main variants.
- Gas numbers were hard to compare.
- Browser path not suitable for 1 GiB benchmarks.

My project roadmap



The variants: who pays and when?

Before exchange

- normal SOX;
- simSOX / no S-deposit;
- $S = B$;
- $S = V$.

During dispute

- $S_B = B$;
- $S_V = V$;
- fewer repeated challenge loops.

For SHA-256 descriptions

- circuit is determined by $|x|$;
- no generic π_1 proof;
- contract reconstructs the gate.

A useful negative result

First implementation attempt

Put all variants into the same large contracts.

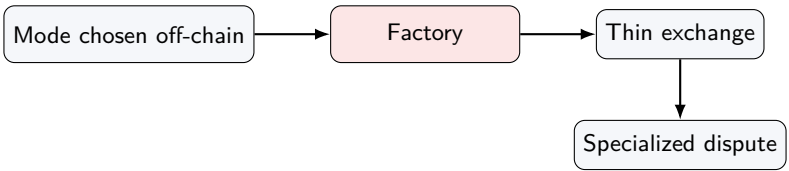
- Easy to validate.
- Bad for gas.
- Every user pays for branches they do not use.

Honest path	Initial	Monolithic
Deployment	2.08M	2.81M
Execution	0.22M	0.23M
Total	2.30M	3.05M

+32.46% gas

This forced the architectural pivot.

Final architecture



Design principle

If a mode is known before execution, do not deploy code for the other modes.

Practical effect

Reusable code is deployed once; each exchange pays only for a small specialized instance.

What I implemented

Area	Contribution
Solidity contracts	Specialized optimistic clones, `SOXFactory`, self-sponsored dispute contracts, hardcoded SHA-256 dispute verifier.
Rust/native tooling	Native large-file precontract and hardcoded dispute witness generation.
Frontend/backend	User-facing mode selection and storage of the selected precontract variant.
Tests and measurements	Same-scope gas benchmarks, 16 KiB dispute variants, complete 1 GiB hardcoded benchmark.
Report methodology	Clear separation between one-time, marginal, dispute, and local runtime costs.

Result 1: honest exchange cost

Initial implementation

Full optimistic account per exchange:

2,300,201 gas

One-time setup

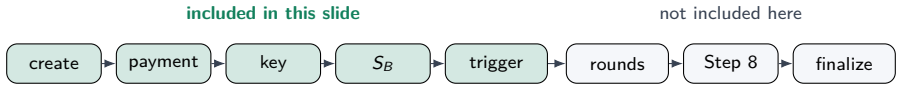
Factory + implementations:

6,584,001 gas

Final mode	Per exchange	Gain
simSOX	500,720	-78.23%
$S = B$	516,349	-77.55%
$S = V$	556,309	-75.81%

about 4x cheaper per new exchange

Result 2a: the baseline comparison window



Architecture	Gas	What changed?
Initial implementation	7,054,365	baseline contract-per-exchange design
Monolithic validator	8,872,096	all variants packed into one large contract
Final normal clone	5,495,184	same normal protocol, cheaper clone architecture

Takeaway

Same validated 4 MiB window in all architectures: -22.10% vs initial. I stop here because the inherited implementation has no equally validated full-dispute baseline in the new variant scopes.

Result 2b: resolving the dispute segment

16 KiB segment $S_B \rightarrow$ End	Monolithic	Final specialized	Gain
Normal, external sponsors	8,345,424	7,432,647	-10.94%
Self-sponsored	7,165,928	5,946,445	-17.02%
Hardcoded SHA-256	8,310,230	5,195,022	-37.49%
Self-sponsored + hardcoded	7,193,323	4,042,185	-43.81%

Takeaway

Different window from the previous slide: this isolates dispute resolution variants, so the baseline is the monolithic validator that implements the same variants.

Hardcoded SHA-256: the most cryptographic optimization

Generic mode

In Step 8, V provides:

- a gate description;
- a proof π_1 that this gate belongs to $h_{circuit}$;
- witness values for local verification.

Hardcoded $desc = \mathrm{SHA256}(x)$

The contract reconstructs the expected gate from public metadata:

- file length;
- description hash;
- AES-CTR IV;
- gate index.

The vendor no longer supplies π_1 or authoritative gate bytes.

Local effect of hardcoding

16 KiB Step 8	Gas	π_1
Generic	463,532	10 items
Hardcoded	400,990	0 items

saving: 62,542 gas

1 GiB check	Generic	Hardcoded
$h_{circuit}$	198,585	29,639–32,390

saving: about 166k–169k gas

Why this is not the whole dispute gain

A full dispute also pays for deployment, challenge rounds, gate evaluation, h_{ct} proofs, *hpre* proofs, and finalization.

Result 3: complete 1 GiB hardcoded dispute

Scale

- 1 GiB input file.
- 16,777,216 ciphertext blocks.
- 33,554,437 gates.
- 26 challenge rounds.

Path component	Gas
Before trigger	2,938,363
Trigger / deploy dispute	2,721,171
After trigger	8,330,558
Complete path	13,990,092

The benchmark reaches the terminal state on a complete 1 GiB path.

Runtime: native path, not browser path

1 GiB measurement	Time	Meaning
Precontract CLI	22.42–48.70 s	read file, encrypt, commit
Dispute data generation	16.43–35.46 s	native benchmark preparation
$hpre_i$ generation	9.47–11.92 s	26 buyer challenge responses
Local Hardhat transactions	0.30–0.56 s	local test execution only

Interpretation

The browser/WebAssembly path is useful for demonstrations. The native Rust path is the right path for large-file measurement.

How I validated the claims

Functional validation

- local end-to-end optimistic flow;
- buyer acceptance and payment;
- vendor key release;
- final contract completion.

Experimental validation

- transaction-receipt gas measurements;
- same-scope benchmark comparisons;
- specialized variant tests;
- complete native 1 GiB hardcoded benchmark.

The report numbers are backed by executable tests, not manual estimates.

Limitations I identified

Prototype limitations

- local accounts, not real wallets;
- backend remains an availability dependency;
- optimized clone paths are measured directly, not fully wallet-integrated.

Cryptographic engineering issues

- inherited gate encoding ambiguities;
- missing Merkle domain separation;
- verification workflow not yet a full production policy.

These are not hidden in the measurements; they define the boundary of the prototype.

Future work

1. **Registry or factory-first architecture**
   Avoid deploying one exchange account per exchange.
2. **Step 1+2 fusion for $S \neq B$**
   Let B bring payment, precontract data, and S's signature in one action.
3. **Cleaner circuit encoding**
   Explicit arity or separate opcodes for ambiguous gates.
4. **Merkle domain separation**
   Separate hash domains for leaves and internal nodes.
5. **Production wallet and verification flow**
   Make every actor see and sign exactly what the protocol requires.

Takeaways

1. The proof of concept was made measurable and variant-aware.

The project reconstructed the stack, mapped it to the protocol, and added reproducible tests.

2. Specialization is the main gas lesson.

The honest exchange drops from 2.30M gas to 0.50–0.56M marginal gas.

3. Large-file execution is feasible with native tooling.

The final benchmark executes a complete hardcoded 1 GiB dispute path up to termination.

Thank you.

Backup: why not always hardcode SHA-256?

Short answer

Hardcoding is correct only when the public description is exactly $desc = \mathrm{SHA256}(x)$.

- For arbitrary predicates, the generic circuit commitment is still needed.
- Hardcoding is a specialization, not a general proof system.
- It is valuable because SHA-256 descriptions are a realistic and important case.

Backup: why 1 GiB timings decrease per round

Observation

The first $hpre_i$ computations are slower than later ones.

- The benchmark forces the binary search to the left.
- The challenge index is roughly divided by two each round.
- The native code recomputes a smaller prefix/accumulator later.
- This affects local CPU time, not on-chain gas.

Backup: what is paid once and what is paid per exchange?

Paid once

Factory and optimistic implementations:

6,584,001 gas

Paid per new exchange

Final optimistic clone modes:

500,720–556,309 gas